

Foxconn Open BMC 新人訓

L2 - D-Bus/Service

Sonny Chu
2025/12/04

環境設定

系統資訊

項目	內容
作業系統	Ubuntu 22.04.5 LTS (VirtualBox)
核心版本	6.8.0-87-generic
Python	3.10.12
硬碟空間	1TB(約 750GB 可用)

OpenBMC 環境

項目	內容
OpenBMC 版本	2.18.0 (commit: 05f7a3f2ed)
MACHINE	romulus
原始碼路徑	~/openbmc
Build 路徑	~/openbmc/build/romulus
映像檔路徑	~/openbmc/build/romulus/tmp/deploy/images/romulus/

作業資訊

作業目標

Level	難度	說明
Level 1	Basic	建立 D-Bus Service, 包含 objectpath, interface, properties
Level 2	Basic	Service 自動監控檔案數值並更新 Property
Level 3	Advanced	當數值超過 critical value 時, 發送 Signal 警報

D-Bus 服務配置

項目	設定值
Service Name	xyz.openbmc_project.SonnyChu.L2Service
Object Path	/xyz/openbmc_project/sonnychu/l2service
Interface Name	xyz.openbmc_project.SonnyChu.Value
監控檔案	/tmp/sensor_value
監控間隔	5 秒
臨界閾值	50

D-Bus Properties

Property	類型	存取	說明
Value	double	可讀可寫	目前監控數值(初始值 25)
Description	string	唯讀	服務說明文字
CriticalThreshold	double	可讀可寫	臨界閾值(初始值 50)

實作內容

Part 1: 程式碼開發

步驟 1: 程式碼結構 - 標頭檔與常數定義

目的: 定義 D-Bus 命名空間、監控設定與全域變數。

```
#include <sdbusplus/asio/connection.hpp>
#include <sdbusplus/asio/object_server.hpp>
#include <sdbusplus/bus.hpp>
#include <boost/asio.hpp>
#include <iostream>
#include <fstream>
#include <string>
#include <chrono>

// =====
// D-Bus 命名常數 (D-Bus Naming Constants)
// =====
constexpr const char* SERVICE_NAME = "xyz.openbmc_project.SonnyChu.L2Service";
constexpr const char* OBJECT_PATH = "/xyz/openbmc_project/sonnychu/l2service";
constexpr const char* INTERFACE_NAME = "xyz.openbmc_project.SonnyChu.Value";

// =====
// 監控設定 (Monitoring Configuration)
// =====
constexpr const char* MONITOR_FILE = "/tmp/sensor_value"; // 模擬 sensor 檔案
constexpr int MONITOR_INTERVAL_SEC = 5; // 監控間隔 (秒)

// =====
// 全域變數 (Global Variables)
// =====
static double currentValue = 25.0; // 目前數值
static double criticalThreshold = 50.0; // 臨界閾值
static std::string description = "SonnyChu L2 Training Service";
static bool alarmActive = false; // 警報狀態

// Interface 指標 (用於發送 signal)
static std::shared_ptr<sdbusplus::asio::dbus_interface> valueInterface;
```

圖 1: 標頭檔引用與常數定義

備註: 使用 sdbusplus 和 boost::asio 函式庫實現非同步 D-Bus 服務。

步驟 2: Level 2 實作 - 檔案讀取函數

目的: 實作 readValueFromFile() 函數, 從監控檔案讀取數值。

```
// =====
// Level 2: 讀取監控檔案的數值
// =====
double readValueFromFile()
{
    std::ifstream file(MONITOR_FILE);
    if (file.is_open())
    {
        double value;
        if (file >> value)
        {
            std::cout << "[Level 2] Read value from file: " << value << std::endl;
            return value;
        }
    }
    return currentValue; // 檔案不存在時, 返回目前值
}
```

圖 2: readValueFromFile() 函數實作

備註: 當檔案不存在或讀取失敗時, 返回目前值避免服務中斷。

步驟 3: Level 3 實作 - Signal 發送函數

目的: 實作 sendThresholdAlarm() 函數, 當數值超過閾值時發送 D-Bus Signal。

```
// =====
// Level 3: 發送 ThresholdAlarm Signal
// 參考: GPUMonitor.cpp 的 signal 發送方式
// =====
void sendThresholdAlarm(double value, double threshold, bool isAssert)
{
    if (valueInterface)
    {
        try
        {
            // Step 1: 創建 signal message
            sdbusplus::message_t msg = valueInterface->new_signal("ThresholdAlarm");

            // Step 2: 添加參數 (value, threshold, isAssert)
            msg.append(value, threshold, isAssert);

            // Step 3: 發送 signal
            msg.signal_send();

            std::cout << "[Level 3] Signal sent - ThresholdAlarm("
                << "value=" << value
                << ", threshold=" << threshold
                << ", isAssert=" << (isAssert ? "true" : "false")
                << ")" << std::endl;
        }
        catch (const std::exception& e)
        {
            std::cerr << "[Level 3] Failed to send signal: " << e.what() << std::endl;
        }
    }
}
```

圖 3: sendThresholdAlarm() 函數實作

Signal 發送三步驟:

```
// Step 1: 創建 signal message
sdbusplus::message_t msg = valueInterface->new_signal("ThresholdAlarm");
// Step 2: 添加參數 (value, threshold, isAssert)
msg.append(value, threshold, isAssert);
// Step 3: 發送 signal
msg.signal_send();
```

備註: 參考 GPUMonitor.cpp 的 signal 發送方式實作。

步驟 4: Level 2 & 3 整合 - 監控函數

目的: 實作 monitorFile() 函數, 整合檔案監控與閾值檢查。

```

// =====
// Level 2 & 3: 定時監控檔案並檢查閾值
// =====
void monitorFile(boost::asio::steady_timer& timer,
                 std::shared_ptr<sdbusplus::asio::dbus_interface> iface)
{
    // Level 2: 讀取檔案並更新 Property
    double newValue = readValueFromFile();

    if (newValue != currentValue)
    {
        currentValue = newValue;
        iface->set_property("Value", currentValue);
        std::cout << "[Level 2] Property updated: Value = " << currentValue << std::endl;
    }

    // Level 3: 檢查是否超過閾值，發送對應的 Signal
    if (currentValue > criticalThreshold && !alarmActive)
    {
        // 超過閾值 → 發送警報 (isAssert = true)
        alarmActive = true;
        sendThresholdAlarm(currentValue, criticalThreshold, true);
    }
    else if (currentValue <= criticalThreshold && alarmActive)
    {
        // 回到正常 → 解除警報 (isAssert = false)
        alarmActive = false;
        sendThresholdAlarm(currentValue, criticalThreshold, false);
    }

    // 設定下次監控時間
    timer.expires_after(std::chrono::seconds(MONITOR_INTERVAL_SEC));
    timer.async_wait([&timer, iface](const boost::system::error_code& ec) {
        if (!ec)
        {
            monitorFile(timer, iface);
        }
    });
}

```

圖 4: `monitorFile()` 函數 - 整合 Level 2 & 3

此函數每 5 秒執行一次，負責讀取檔案、更新 Property、檢查閾值並發送警報。

步驟 5: Level 1 實作 - D-Bus 初始化與 Property 註冊

目的: 在 main() 函數中初始化 D-Bus 連線並註冊 Properties。

```
// =====
// Main Function
// =====
int main()
{
    // 顯示啟動資訊
    std::cout << "===== " << std::endl;
    std::cout << "  SonnyChu L2 Training Service " << std::endl;
    std::cout << "===== " << std::endl;
    std::cout << "Bus Name: " << SERVICE_NAME << std::endl;
    std::cout << "Object Path: " << OBJECT_PATH << std::endl;
    std::cout << "Interface: " << INTERFACE_NAME << std::endl;
    std::cout << "Monitor: " << MONITOR_FILE << " (every " << MONITOR_INTERVAL_SEC << "s)" << std::endl;
    std::cout << "Threshold: " << criticalThreshold << std::endl;
    std::cout << "===== " << std::endl;

    // 建立 I/O context 和 D-Bus connection
    boost::asio::io_context io;
    auto conn = std::make_shared<sdbusplus::asio::connection>(io);

    // 註冊 Bus Name
    conn->request_name(SERVICE_NAME);

    // 建立 Object Server
    sdbusplus::asio::object_server server(conn);

    // =====
    // Level 1: 創建 Interface 並註冊 Properties
    // =====
    valueInterface = server.add_interface(OBJECT_PATH, INTERFACE_NAME);

    // Property: Value (可讀可寫, emits-change)
    valueInterface->register_property(
        "Value", currentValue,
        // Setter: 當外部修改 Value 時觸發
        [](const double& newValue, double& value) {
            std::cout << "[Level 1] Property Value changed: " << value << " -> " << newValue << std::endl;
            value = newValue;
            return true;
        },
        // Getter: 讀取 Value 時觸發
        [](const double& value) {
            return value;
        }
    );
};
```

圖 5: main() 函數 - D-Bus 初始化

```
// Property: Description (唯讀)
valueInterface->register_property_r(
    "Description", description,
    sdbusplus::vtable::property::const_,
    [](const std::string& desc) {
        return desc;
    }
);

// Property: CriticalThreshold (可讀可寫)
valueInterface->register_property(
    "CriticalThreshold", criticalThreshold,
    [](const double& newValue, double& value) {
        std::cout << "[Level 1] CriticalThreshold changed: " << value << " -> " << newValue << std::endl;
        value = newValue;
        criticalThreshold = newValue; // 同步更新全域變數
        return true;
    },
    [](const double& value) {
        return value;
    }
);
```

圖 6: main() 函數 - Properties 註冊

步驟 6: Signal 註冊與 Timer 啟動

目的: 註冊 ThresholdAlarm Signal 並啟動定時監控。

```
// =====
// Level 3: 註冊 Signal (double value, double threshold, bool isAssert)
// =====
valueInterface->register_signal<double, double, bool>("ThresholdAlarm");

// 初始化 Interface
valueInterface->initialize();

std::cout << "[Level 1] D-Bus Service initialized successfully!" << std::endl;

// =====
// Level 2: 啟動檔案監控 (每 5 秒讀取一次)
// =====
boost::asio::steady_timer monitorTimer(io);
monitorTimer.expires_after(std::chrono::seconds(MONITOR_INTERVAL_SEC));
monitorTimer.async_wait([&monitorTimer](const boost::system::error_code& ec) {
    if (!ec)
    {
        monitorFile(monitorTimer, valueInterface);
    }
});

std::cout << "[Level 2] File monitoring started!" << std::endl;
std::cout << "[Level 3] ThresholdAlarm signal registered!" << std::endl;
std::cout << "Service is running..." << std::endl;

// 開始事件迴圈
io.run();

return 0;
}
```

圖 7: main() 函數 - Signal 註冊與 Timer 啟動

✓ 完成: 主程式 sonnychu-l2-service.cpp 開發完成。

Part 2: 編譯配置

步驟 7: Meson 建置檔案

目的: 配置 meson.build, 設定 C++23 標準以支援 sdbusplus。

```
sonny@sonny-VirtualBox:~$ cat ~/openbmc/meta-bmc-training/recipes-l2/sonnychu-l2
-service/files/meson.build
project(
    'sonnychu-l2-service',
    'cpp',
    version: '1.0',
    default_options: [
        'cpp_std=c++23',
        'warning_level=3',
    ],
)

sdbusplus_dep = dependency('sdbusplus')
boost_dep = dependency('boost')

executable(
    'sonnychu-l2-service',
    'sonnychu-l2-service.cpp',
    dependencies: [sdbusplus_dep, boost_dep],
    install: true,
    install_dir: '/usr/bin',
)
```

圖 8: meson.build 配置

備註: OpenBMC v2.18.0 的 sdbusplus 使用 std::move_only_function, 需要 C++23 支援。

步驟 8: BitBake Recipe

目的: 建立 .bb 檔案, 整合到 OpenBMC build 系統。

```
sonny@sonny-VirtualBox:~$ cat ~/openbmc/meta-bmc-training/recipes-l2/sonnychu-l2
-service/sonnychu-l2-service_1.0.bb
SUMMARY = "SonnyChu L2 Training D-Bus Service"
DESCRIPTION = "A D-Bus service for OpenBMC L2 training with file monitoring and
threshold alarm"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda
2f7b4f302"

SRC_URI = "file://sonnychu-l2-service.cpp \
           file://meson.build \
           file://sonnychu-l2-service.service"

S = "${WORKDIR}/sources-unpack"

inherit meson pkgconfig systemd

DEPENDS = "sdbusplus boost"

SYSTEMD_SERVICE:${PN} = "sonnychu-l2-service.service"
SYSTEMD_AUTO_ENABLE = "enable"

do_install:append() {
    install -d ${D}${systemd_system_unitdir}
    install -m 0644 ${S}/sonnychu-l2-service.service ${D}${systemd_system_unitdi
r}/
}

FILES:${PN} += "${systemd_system_unitdir}/sonnychu-l2-service.service"
```

圖 9: BitBake Recipe 配置

重要設定：

```

S = "${WORKDIR}/sources-unpack" # 新版 Yocto 要求
inherit meson pkgconfig systemd # 必須繼承這三個 class
DEPENDS = "sdbusplus boost" # 相依套件

```

步驟 9: Systemd Service 檔案

目的: 配置 systemd service, 使服務開機自動啟動。



```

sonny@sonny-VirtualBox:~$ cat ~/openbmc/meta-bmc-training/recipes-l2/sonnychu-l2
-service/files/sonnychu-l2-service.service
[Unit]
Description=SonnyChu L2 Training D-Bus Service
After=dbus.service
Requires=dbus.service

[Service]
Type=simple
ExecStart=/usr/bin/sonnychu-l2-service
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target

```

圖 10: systemd service 配置

✓ 完成: 編譯配置完成, 可以開始 bitbake 編譯。

Part 3: 測試驗證

步驟 10: 驗證服務運行狀態

目的: 確認 D-Bus Service 在 QEMU 環境中正常運行。

```
systemctl status sonnychu-l2-service
```

```

root@romulus:~# systemctl status sonnychu-l2-service
* sonnychu-l2-service.service - SonnyChu L2 Training D-Bus Service
   Loaded: loaded (/usr/lib/systemd/system/sonnychu-l2-service.service; enable
   #; preset: enabled)
   Active: active (running) since Thu 2025-12-04 06:47:59 UTC; 3min 59s ago
   Invocation: c835876db084537b64867362ee3691e
   Main PID: 253 (sonnychu-l2-ser)
   CPU: 242ms
   CGroup: /system.slice/sonnychu-l2-service.service
           -253 /usr/bin/sonnychu-l2-service

Dec 04 06:48:01 romulus sonnychu-l2-service[253]: Bus Name: xyz.openbmc...ice
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: Object Path: /xyz/openbm...ice
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: Interface: xyz.openbmc...lue
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: Monitor: /tmp/sensor...5s)
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: Threshold: 50
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: =====
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: [Level 1] D-Bus Service ...lyl
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: [Level 2] File monitorln...ed!
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: [Level 3] Thresholdalarm...ed!
Dec 04 06:48:01 romulus sonnychu-l2-service[253]: Service is running...
Hint: Some lines were ellipsized, use -l to show in full.
root@romulus:~#

```

圖 11: Service 運行狀態 - Level 1, 2, 3 全部初始化成功

步驟 11: Level 1 測試 - D-Bus Interface 驗證

目的: 使用 busctl introspect 驗證 D-Bus Interface 結構。

```
busctl introspect xyz.openbmc_project.SonnyChu.L2Service \
/xyz/openbmc_project/sonnychu/l2service
```

```

root@romulus:~# busctl introspect xyz.openbmc_project.SonnyChu.L2Service /xyz/op
enbmc_project/sonnychu/l2service
NAME                                TYPE      SIGNATURE RESULT/VALUE
--                                -
org.freedesktop.DBus.Introspectable interface -
-
-Introspect                        method   -         s
-
org.freedesktop.DBus.Peer          interface -
-
-GetMachineId                     method   -         s
-
-Ping                             method   -         -
-
org.freedesktop.DBus.Properties    interface -
-
-Get                              method   ss        v
-
-GetAll                           method   s         a{sv}
-
-Set                              method   ssv       -
-
-PropertiesChanged                 signal   sa{sv}as  -
-
xyz.openbmc_project.SonnyChu.Value interface -
-
-CriticalThreshold                 property d        50
  emits-change writable
-Description                       property s        "SonnyChu L2 Training Se
rvice" const
-Value                            property d        25
  emits-change writable
-ThresholdAlarm                   signal   ddb       -
-
root@romulus:~#

```

圖 12: D-Bus Interface 結構 - 顯示 Properties 與 Signal

✓ 完成: Level 1 驗證通過 - Interface、Properties、Signal 皆已註冊。

步驟 12: Level 2 測試 - 檔案自動監控

目的: 驗證 Service 能自動監控檔案並更新 Property。

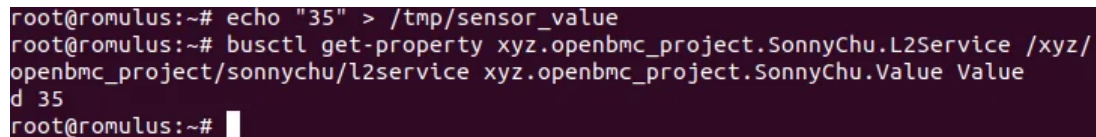
測試步驟:

1. 寫入測試數值到監控檔案

```
echo "35" > /tmp/sensor_value
```

2. 等待 5 秒後查詢 Property

```
busctl get-property xyz.openbmc_project.SonnyChu.L2Service \
  /xyz/openbmc_project/sonnychu/l2service \
  xyz.openbmc_project.SonnyChu.Value Value
```



```
root@romulus:~# echo "35" > /tmp/sensor_value
root@romulus:~# busctl get-property xyz.openbmc_project.SonnyChu.L2Service /xyz/
openbmc_project/sonnychu/l2service xyz.openbmc_project.SonnyChu.Value Value
d 35
root@romulus:~#
```

圖 13: Level 2 測試 - Value 從 25 自動更新為 35

✓ 完成: Level 2 驗證通過 - Service 成功自動監控檔案並更新 Property。

步驟 13: Level 3 測試 - Signal 警報觸發

目的: 驗證當數值超過閾值時, Service 自動發送 ThresholdAlarm Signal。

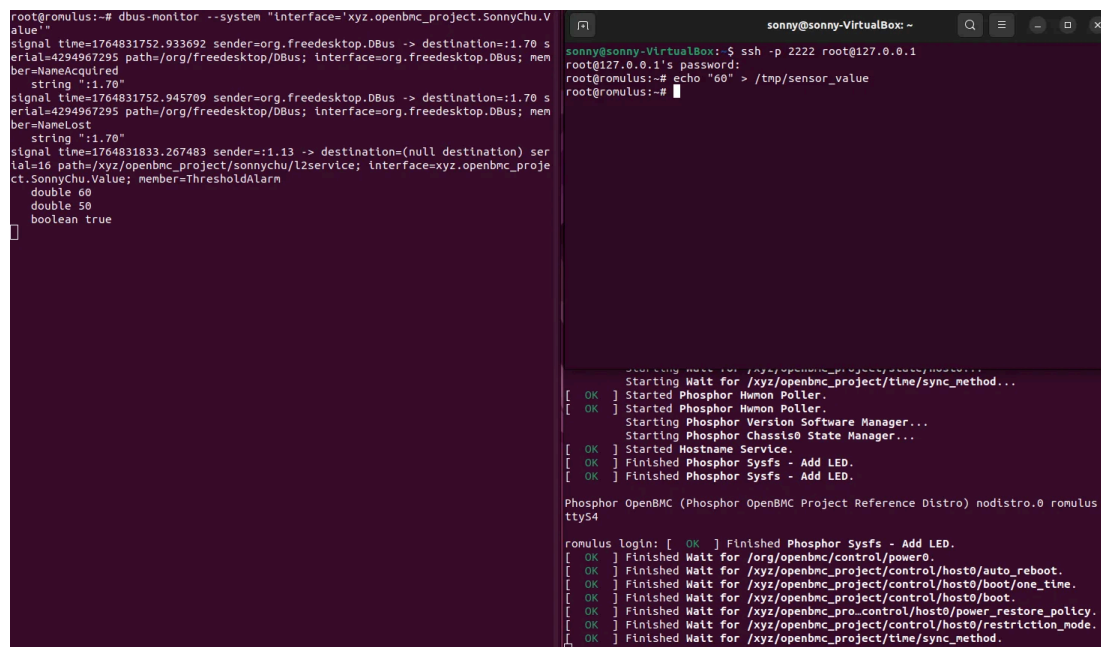
測試步驟:

終端 1: 啟動 dbus-monitor 監聽

```
dbus-monitor --system "interface='xyz.openbmc_project.SonnyChu.Value'"
```

終端 2: 寫入超過閾值的數值

```
echo "60" > /tmp/sensor_value
```



```
root@romulus:~# dbus-monitor --system "interface='xyz.openbmc_project.SonnyChu.V
alue'"
signal time=1764831752.933692 sender=org.freedesktop.DBus -> destination=:1.70 s
erial=4294967295 path=/org/freedesktop/DBus; interface=org.freedesktop.DBus; mem
ber=NameAcquired
    string ":1.70"
signal time=1764831752.945709 sender=org.freedesktop.DBus -> destination=:1.70 s
erial=4294967295 path=/org/freedesktop/DBus; interface=org.freedesktop.DBus; mem
ber=NameLost
    string ":1.70"
signal time=1764831833.267483 sender=:1.13 -> destination=(null destination) ser
ial=10 path=/xyz/openbmc_project/sonnychu/l2service; interface=xyz.openbmc_proje
ct.SonnyChu.Value; member=ThresholdAlarm
    double 60
    double 50
    boolean true
[]

sonny@sonny-VirtualBox: ~$ ssh -p 2222 root@127.0.0.1
root@127.0.0.1's password:
root@romulus:~# echo "60" > /tmp/sensor_value
root@romulus:~#
```

圖 14: dbus-monitor 捕獲 ThresholdAlarm Signal (isAssert=true)

Signal 內容解析:

member=ThresholdAlarm

double 60 # 當前數值

```
double 50      # 臨界閾值
boolean true   # 警報狀態 (true = 觸發)
```

步驟 14: Level 3 測試 - 警報解除

目的: 驗證當數值回到正常範圍時, Service 發送 isAssert=false 的 Signal。

```
echo "40" > /tmp/sensor_value
```

```
root@romulus:~# dbus-monitor --system "interface='xyz.openbmc_project.SonnyChu.V1'"
signal time=1764831752.933692 sender=org.freedesktop.DBus -> destination=:1.70 s
erial=4294967295 path=/org/freedesktop/DBus; interface=org.freedesktop.DBus; men
ber=NameAcquired
    string ":1.70"
signal time=1764831752.945709 sender=org.freedesktop.DBus -> destination=:1.70 s
erial=4294967295 path=/org/freedesktop/DBus; interface=org.freedesktop.DBus; men
ber=NameLost
    string ":1.70"
signal time=1764831833.267483 sender=:1.13 -> destination=(null destination) ser
ial=18 path=/xyz/openbmc_project/sonnychu/l2service; interface=xyz.openbmc_proje
ct.SonnyChu.Value; member=ThresholdAlarm
    double 60
    double 50
    boolean true
signal time=1764833421.191724 sender=:1.13 -> destination=(null destination) ser
ial=18 path=/xyz/openbmc_project/sonnychu/l2service; interface=xyz.openbmc_proje
ct.SonnyChu.Value; member=ThresholdAlarm
    double 40
    double 50
    boolean false

[ OK ] Started Phosphor Hwmon Poller.
[ OK ] Started Phosphor Hwmon Poller.
Starting Phosphor Version Software Manager...
Starting Phosphor Chassis0 State Manager...
[ OK ] Started Hostname Service.
[ OK ] Finished Phosphor Sysfs - Add LED.
[ OK ] Finished Phosphor Sysfs - Add LED.

Phosphor OpenBMC (Phosphor OpenBMC Project Reference Distro) nodistro.0 romulus
ttyS4

romulus login: [ OK ] Finished Phosphor Sysfs - Add LED.
[ OK ] Finished Wait for /org/openbmc/control/power0.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/auto_reboot.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/boot/one_time.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/boot.
[ OK ] Finished Wait for /xyz/openbmc_pro. control/host0/power_restore_policy.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/restriction_mode.
[ OK ] Finished Wait for /xyz/openbmc_project/time/sync_method.
```

圖 15: 完整警報流程 - 觸發 (true) → 解除 (false)

✓ 完成: Level 3 驗證通過 - Signal 警報機制運作正常。

結論

完成項目

Level	難度	實作內容	狀態
Level 1	Basic	D-Bus Service 含 objectpath, interface, properties	✓ 完成
Level 2	Basic	自動監控檔案數值並更新 Property	✓ 完成
Level 3	Advanced	超過閾值時發送 ThresholdAlarm Signal	✓ 完成

學習心得

透過本次 L2 作業，學習到以下重點：

1. D-Bus 服務架構：了解 sdbusplus 函式庫的使用方式，包含 Property 註冊（可讀可寫、唯讀）與 Signal 發送機制。
2. 非同步程式設計：使用 boost::asio 實現定時器功能，配合 io_context 事件迴圈處理非同步操作。
3. OpenBMC Build System：熟悉 BitBake recipe 撰寫、meson.build 配置，以及 systemd service 整合。
4. 除錯技巧：學會使用 busctl introspect 檢視 D-Bus 結構、dbus-monitor 監聽 Signal、systemctl 管理服務狀態。

特別注意 OpenBMC v2.18.0 的技術要點：

```
# meson.build 需設定 C++23 (sdbusplus 使用 std::move_only_function)
cpp_std=c++23

# BitBake recipe 不能使用 S = "${WORKDIR}"
S = "${WORKDIR}/sources-unpack"

# 必須繼承 pkgconfig 以找到 sdbusplus
inherit meson pkgconfig systemd
```